

API REST

Campus Digital

Documentación Técnica Completa
Versión 1.0 · 2026

Módulos documentados	12
Endpoints totales	80+
Casos de prueba	120+
Formato de datos	JSON / REST

Documentación Técnica de API REST

Campo	Valor
Base URL	http://localhost:8000/api
Versión	1.0.0
Formato	JSON (application/json)
Autenticación	API Key via header X-API-KEY
Paginación	15 registros por página (configurable con ?per_page=N)

Autenticación

Todas las solicitudes (excepto POST /api/rfid/auth) deben incluir el siguiente encabezado HTTP:

```
X-API-KEY: 4f8a2b1c9d3e7f6a0b5c8d2e1f4a7b3c6d9e2f5a8b1c4d7e0f3a6b9c2d5e8f1a
Accept: application/json
```

Códigos de Respuesta Globales

Código	Descripción
200 OK	Solicitud exitosa, respuesta con datos
201 Created	Recurso creado exitosamente
204 No Content	Operación exitosa sin cuerpo de respuesta
400 Bad Request	Parámetros inválidos o faltantes en el body
401 Unauthorized	API Key ausente o inválida
403 Forbidden	Sin permisos para ejecutar esta acción
404 Not Found	Recurso no encontrado
422 Unprocessable Entity	Regla de negocio violada (ej. saldo insuficiente)
500 Internal Server Error	Error interno del servidor

1. Módulo: Usuarios

Gestión completa de usuarios del sistema: creación, lectura, actualización, baja lógica y control de bloqueo de cuentas.

Endpoints

Método	URL	Descripción
GET	/usuarios	Lista paginada de usuarios activos
GET	/usuarios?search=juan	Búsqueda por nombre, apellido o email
GET	/usuarios?bloqueado=true	Filtrar sólo usuarios bloqueados
GET	/usuarios/{id}	Detalle de un usuario específico
POST	/usuarios	Crear nuevo usuario
PUT	/usuarios/{id}	Actualizar datos del usuario
DELETE	/usuarios/{id}	Baja lógica del usuario
POST	/usuarios/{id}/toggle-block	Bloquear / desbloquear cuenta

Parámetros de Consulta

Parámetro	Tipo	Descripción
search	string	Texto a buscar en nombre, apellido o email
bloqueado	boolean	true = cuentas bloqueadas · false = cuentas activas
per_page	integer	Número de registros por página (default 15)
page	integer	Número de página para paginación

Ejemplos de Body (JSON)

POST /usuarios – Crear usuario

```
{
  "nombre": "Carlos",
  "apellido": "Lopez",
```

```
{
  "email": "carlos@test.com",
  "password": "12345678",
  "telefono": "5551234567"
}
```

PUT /usuarios/{id} – Actualizar usuario

```
{
  "nombre": "Carlos Alberto",
  "telefono": "5559999999"
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta campo requerido (nombre, email, password)	{ "message": "El campo email es obligatorio" }
401	Header X-API-KEY ausente o inválido	{ "message": "Unauthorized" }
404	ID de usuario no existe	{ "message": "Usuario no encontrado" }
422	Email ya registrado en el sistema	{ "message": "El email ya está en uso" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /usuarios – Sin API Key	401 Unauthorized	Verificar header X-API-KEY requerido
GET /usuarios – Con API Key válida	200 + lista paginada	Confirmar estructura { data, total, per_page }
GET /usuarios?bloqueado=true	200 usuarios filtrados	Solo retorna usuarios con bloqueado=true
GET /usuarios/999 – ID inexistente	404 Not Found	Mensaje claro de recurso no encontrado
POST /usuarios – Body completo	201 Created	Retorna el usuario creado con su ID
POST /usuarios – Email duplicado	422	Error indicando que el email ya existe

POST /usuarios – Sin campo password	400 Bad Request	Validation error en campo password
PUT /usuarios/1 – Solo telefono	200 OK	Solo actualiza el campo enviado
DELETE /usuarios/1	200 o 204	Soft delete; usuario no aparece en GET /usuarios
POST /usuarios/1/toggle-block activo	200 bloqueado=true	Estado cambia a bloqueado
POST /usuarios/1/toggle-block bloqueado	200 bloqueado=false	Estado regresa a activo

2. Módulo: Roles

Administración de roles del sistema (Administrador, Estudiante, Proveedor, etc.). Cada rol agrupa permisos que controlan el acceso a funcionalidades.

Endpoints

Método	URL	Descripción
GET	/roles	Lista de todos los roles del sistema
GET	/roles/{id}	Detalle de rol con sus permisos asignados
POST	/roles	Crear nuevo rol
PUT	/roles/{id}	Actualizar nombre, descripción o estado
DELETE	/roles/{id}	Baja lógica del rol

Ejemplos de Body (JSON)

POST /roles – Crear rol

```
{
  "nombre": "moderador",
  "descripcion": "Rol de moderación de contenido",
  "activo": true
}
```

PUT /roles/{id} – Desactivar rol

```
{
  "descripcion": "Descripción actualizada",
  "activo": false
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta campo nombre al crear	{ "message": "El campo nombre es obligatorio" }
404	ID de rol no existe	{ "message": "Rol no encontrado" }
422	Nombre de rol ya registrado	{ "message": "El nombre del rol ya existe" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /roles – listar todos	200 + array de roles	Confirmar array con id, nombre, descripcion, activo
GET /roles/1 – con permisos	200 + permisos	Verificar que incluye relación de permisos
GET /roles/999 – inexistente	404	Mensaje de rol no encontrado
POST /roles – body válido	201 Created	Rol creado con ID asignado
POST /roles – sin nombre	400 Bad Request	Error de validación en campo nombre
PUT /roles/1 – desactivar	200 activo=false	Confirmar que activo cambió a false
DELETE /roles/1	200 o 204	Rol no aparece en GET /roles activos

3. Módulo: Permisos

Catálogo de permisos granulares (user.read, user.write, audit.read, etc.) que se asignan a roles para controlar el acceso a cada funcionalidad del sistema.

Endpoints

Método	URL	Descripción
GET	/permisos	Lista de todos los permisos
GET	/permisos/{id}	Detalle de un permiso específico
POST	/permisos	Crear nuevo permiso
PUT	/permisos/{id}	Actualizar descripción o estado
DELETE	/permisos/{id}	Baja lógica del permiso

Ejemplos de Body (JSON)

POST /permisos – Crear permiso

```
{
  "clave": "reporte.read",
  "descripcion": "Permite visualizar reportes del sistema",
  "activo": true
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta campo clave	{ "message": "El campo clave es obligatorio" }
404	Permiso no encontrado	{ "message": "Permiso no encontrado" }
422	Clave de permiso duplicada	{ "message": "La clave ya está registrada" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /permisos	200 array de permisos	Incluye id, clave, descripcion, activo
GET /permisos/999	404	Recurso no encontrado
POST /permisos – clave nueva	201 Created	Permiso creado correctamente
POST /permisos – clave dup.	422	Error: clave ya registrada
PUT /permisos/1	200 OK	Descripción actualizada
DELETE /permisos/1	200 o 204	Soft delete confirmado

4. Módulo: Rol-Permisos (Asignación)

Gestión de la relación muchos-a-muchos entre roles y permisos. Permite consultar qué permisos tiene cada rol y agregar o quitar capacidades.

Endpoints

Método	URL	Descripción
GET	/rol-permisos	Todas las asignaciones rol-permiso
GET	/rol-permisos?rol_id=1	Permisos del rol con ID 1
POST	/rol-permisos	Asignar permiso a un rol
DELETE	/rol-permisos/{id}	Quitar permiso de un rol

Parámetros de Consulta

Parámetro	Tipo	Descripción
rol_id	integer	Filtra asignaciones de un rol específico

Ejemplos de Body (JSON)

POST /rol-permisos – Asignar permiso

```
{
  "rol_id": 1,
  "permiso_id": 2
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta rol_id o permiso_id	{ "message": "Los campos rol_id y permiso_id

		son obligatorios" }
404	El rol o permiso no existe	{ "message": "Rol o permiso no encontrado" }
422	La asignación ya existe	{ "message": "Este permiso ya está asignado al rol" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /rol-permisos?rol_id=1	200 lista de permisos	Retorna permisos del rol 1
GET /rol-permisos?rol_id=999	200 array vacío	Array vacío si no tiene permisos
POST – asignación nueva	201 Created	Registro de asignación creado
POST – asignación duplicada	422	Error de asignación existente
DELETE /rol-permisos/1	200 o 204	Permiso removido del rol
DELETE /rol-permisos/999	404	Registro de asignación no existe

5. Módulo: Usuario-Roles (Asignación)

Gestión de la relación entre usuarios y roles. Determina qué nivel de acceso tiene cada usuario dentro del sistema.

Endpoints

Método	URL	Descripción
GET	/usuario-roles	Todas las asignaciones usuario-rol
GET	/usuario-roles?usuario_id=1	Roles del usuario ID 1
GET	/usuario-roles?rol_id=1	Usuarios con el rol ID 1
POST	/usuario-roles	Asignar rol a un usuario
DELETE	/usuario-roles/{id}	Quitar rol a un usuario

Parámetros de Consulta

Parámetro	Tipo	Descripción
usuario_id	integer	Filtra roles de un usuario específico
rol_id	integer	Filtra usuarios que tienen un rol específico

Ejemplos de Body (JSON)

POST /usuario-roles – Asignar rol

```
{
  "usuario_id": 4,
  "rol_id": 2
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta usuario_id o rol_id	{ "message": "Los campos son obligatorios" }
404	Usuario o rol no existe	{ "message": "Recurso no encontrado" }
422	Asignación ya existe	{ "message": "El usuario ya tiene este rol" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET ?usuario_id=4	200 roles del usuario	Lista de roles asignados
GET ?rol_id=1	200 usuarios con rol	Lista de usuarios con ese rol
POST – asignación válida	201 Created	Registro creado correctamente
POST – duplicada	422	Error de duplicado
DELETE /usuario-roles/1	200 o 204	Rol quitado del usuario

6. Módulo: Perfiles de Usuario

Datos personales extendidos del usuario: fecha de nacimiento, género y dirección. Complementan la información básica del módulo de usuarios.

Endpoints

Método	URL	Descripción
GET	/usuario-perfiles	Perfiles de todos los usuarios
GET	/usuario-perfiles/{id}	Perfil del usuario con ID dado
PUT	/usuario-perfiles/{id}	Actualizar perfil del usuario

Ejemplos de Body (JSON)

PUT /usuario-perfiles/{id} – Actualizar perfil

```
{
  "fecha_nacimiento": "1995-06-15",
  "genero": "masculino",
  "direccion": "Calle Falsa 123, CDMX"
}
```

Casos de Error

Código	Causa	Respuesta esperada
404	Perfil del usuario no existe	{ "message": "Perfil no encontrado" }
422	Formato de fecha inválido	{ "message": "fecha_nacimiento debe ser YYYY-MM-DD" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /usuario-perfiles/1	200 datos de perfil	Retorna fecha_nacimiento, genero, direccion
GET /usuario-perfiles/999	404	Perfil no encontrado
PUT /usuario-perfiles/1 – válido	200 actualizado	Confirmar campos actualizados
PUT – fecha mal formateada	422	Error de formato de fecha

7. Módulo: Sesiones

Historial de sesiones abiertas y cerradas. Permite auditar desde dónde y cuándo se conectó cada usuario, incluyendo IP y agente de usuario.

Endpoints

Método	URL	Descripción
GET	/sesiones	Todas las sesiones paginadas
GET	/sesiones?activa=true	Solo sesiones activas en este momento
GET	/sesiones?activa=false	Solo sesiones ya cerradas
GET	/sesiones?usuario_id=1	Sesiones del usuario ID 1
GET	/sesiones?desde=X&hasta=Y	Sesiones en rango de fechas
GET	/sesiones/{id}	Detalle completo de una sesión

Parámetros de Consulta

Parámetro	Tipo	Descripción
activa	boolean	true = sesiones abiertas · false = sesiones cerradas
usuario_id	integer	Filtra sesiones de un usuario específico
desde / hasta	date	Rango de fechas en formato YYYY-MM-DD
per_page	integer	Registros por página (default 15)

Casos de Error

Código	Causa	Respuesta esperada
401	Sin API Key	{ "message": "Unauthorized" }
404	Sesión con ID no encontrada	{ "message": "Sesión no encontrada" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /sesiones	200 paginado	Estructura { data, total, per_page, current_page }
GET /sesiones?activa=true	200 solo activas	Todas las sesiones con activa=true
GET /sesiones?usuario_id=1	200 por usuario	Solo sesiones del usuario 1
GET /sesiones?desde=X&hasta=Y	200 rango fechas	Filtro por rango aplicado correctamente
GET /sesiones/1	200 detalle	Incluye IP, navegador, inicio y expiración
GET /sesiones/999	404	Sesión no encontrada

8. Módulo: Bitácora – Accesos

Registro inmutable de eventos de autenticación: login exitoso, login fallido, logout y registro de nuevos usuarios. Fundamental para detectar ataques y auditar el sistema.

Endpoints

Método	URL	Descripción
GET	/bitacora/accesos	Todos los eventos de acceso paginados
GET	/bitacora/accesos?evento=login_success	Solo inicios de sesión exitosos
GET	/bitacora/accesos?evento=login_failed	Solo intentos fallidos
GET	/bitacora/accesos?evento=logout	Solo cierres de sesión
GET	/bitacora/accesos?exito=false	Solo eventos fallidos
GET	/bitacora/accesos?usuario_id=1	Eventos del usuario ID 1
GET	/bitacora/accesos?email=x@y.com	Rastrear intentos por email
GET	/bitacora/accesos?desde=X&hasta=Y	Eventos en rango de fechas
GET	/bitacora/accesos/{id}	Detalle de evento específico

Parámetros de Consulta

Parámetro	Tipo	Descripción
evento	string	login_success · login_failed · logout · registro
exito	boolean	true = exitosos · false = fallidos
usuario_id	integer	Filtra eventos de un usuario
email	string	Filtra por email usado en el intento
desde/hasta	date	Rango de fechas YYYY-MM-DD
per_page	integer	Registros por página

Casos de Error

Código	Causa	Respuesta esperada
401	Sin API Key	{ "message": "Unauthorized" }
404	Registro no encontrado	{ "message": "Registro no encontrado" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /bitacora/accesos	200 paginado	Lista completa de eventos
GET ?evento=login_success	200 solo exitosos	Filtrado por tipo de evento
GET ?evento=login_failed	200 intentos fallidos	Útil para detectar brute force
GET ?exito=false	200 solo fallidos	Todos los eventos no exitosos
GET ?email=admin@campusdigital.com	200 por email	Rastreo de intentos por correo
GET ?usuario_id=1&desde=X&hasta=Y	200 filtro combinado	Filtros simultáneos correctos
GET /bitacora/accesos/7	200 detalle evento	Incluye IP, navegador, resultado
GET /bitacora/accesos/999	404	Evento no encontrado

9. Módulo: Bitácora – Actividad

Registro de acciones realizadas dentro del sistema: creación, actualización, eliminación de usuarios, cambios de estado, etc. Permite trazar quién hizo qué y cuándo.

Endpoints

Método	URL	Descripción
GET	/bitacora/actividad	Todas las acciones del sistema
GET	/bitacora/actividad?usuario_id=1	Acciones del usuario ID 1
GET	/bitacora/actividad?modulo=seguridad	Acciones del módulo seguridad
GET	/bitacora/actividad?accion=eliminar_usuario	Registros de bajas de usuario
GET	/bitacora/actividad?accion=actualizar_usuario	Registros de actualizaciones
GET	/bitacora/actividad?target_tabla=usuario	Acciones sobre tabla usuario
GET	/bitacora/actividad?exito=true	Solo acciones exitosas
GET	/bitacora/actividad?desde=X&hasta=Y	Acciones en rango de fechas
GET	/bitacora/actividad/{id}	Detalle completo de una acción

Parámetros de Consulta

Parámetro	Tipo	Descripción
usuario_id	integer	Filtra por el usuario que realizó la acción
modulo	string	Nombre del módulo (seguridad, cafeteria, etc.)
accion	string	eliminar_usuario · actualizar_usuario · cambiar_estado_usuario
target_tabla	string	Tabla afectada por la acción (usuario, rol, etc.)
exito	boolean	true = acciones exitosas · false = fallidas
desde/hasta	date	Rango de fechas YYYY-MM-DD

Casos de Error

Código	Causa	Respuesta esperada
401	Sin API Key	{ "message": "Unauthorized" }
404	Registro no encontrado	{ "message": "Actividad no encontrada" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /bitacora/actividad	200 paginado	Lista de todas las acciones
GET ?modulo=seguridad	200 filtrado	Solo acciones del módulo seguridad
GET ?accion=eliminar_usuario	200 eliminaciones	Solo registros de bajas
GET ?usuario_id=1&exito=true	200 filtro múltiple	Acciones exitosas del usuario 1
GET ?target_tabla=usuario	200 por tabla	Todo lo que modificó la tabla usuario
GET /bitacora/actividad/1	200 detalle	Incluye qué, quién, sobre qué registro y desde qué IP
GET /bitacora/actividad/999	404	Registro no encontrado

10. Módulo: Tarjetas Universitarias (RFID/NFC)

Administración de las tarjetas físicas RFID/NFC del campus. Incluye registro, bloqueo, desbloqueo y consulta por UID de chip.

Endpoints

Método	URL	Descripción
GET	/tarjetas	Todas las tarjetas del sistema
GET	/tarjetas?estado=activa	Solo tarjetas activas
GET	/tarjetas?estado=bloqueada	Solo tarjetas bloqueadas
GET	/tarjetas?estado=perdida	Solo tarjetas perdidas
GET	/tarjetas?usuario_id=1	Tarjeta(s) del usuario ID 1
GET	/tarjetas/uid/{uid}	Buscar tarjeta por UID del chip
GET	/tarjetas/{id}	Detalle de tarjeta por ID interno
POST	/tarjetas	Registrar nueva tarjeta
PUT	/tarjetas/{id}	Actualizar UID, estado o notas
DELETE	/tarjetas/{id}	Baja lógica de la tarjeta
POST	/tarjetas/{id}/bloquear	Bloquear tarjeta con motivo
POST	/tarjetas/{id}/desbloquear	Desbloquear tarjeta

Parámetros de Consulta

Parámetro	Tipo	Descripción
estado	string	activa · bloqueada · perdida · cancelada
usuario_id	integer	Filtra tarjetas de un usuario específico

Ejemplos de Body (JSON)

POST /tarjetas – Registrar tarjeta

```
{
  "usuario_id": 4,
  "uid": "A1B2C3D4E5F6",
  "estado": "activa",
  "registrado_por_usuario_id": 1
}
```

POST /tarjetas/{id}/bloquear

```
{
  "motivo_bloqueo": "Tarjeta reportada como robada",
  "bloqueado_por_usuario_id": 1
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta uid o usuario_id	{ "message": "Los campos uid y usuario_id son obligatorios" }
404	Tarjeta o UID no encontrado	{ "message": "Tarjeta no encontrada" }
422	UID ya registrado en el sistema	{ "message": "El UID ya pertenece a otra tarjeta" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /tarjetas	200 lista completa	Lista con id, uid, estado, usuario_id
GET ?estado=activa	200 solo activas	Filtro por estado aplicado
GET ?estado=bloqueada	200 solo bloqueadas	Tarjetas suspendidas
GET /tarjetas/uid/A1B2C3D4E5F6	200 con usuario y perfil	Respuesta del lector RFID
GET /tarjetas/uid/INVALIDO	404	UID no registrado
POST - tarjeta nueva	201 Created	Tarjeta creada con ID

POST – UID duplicado	422	Error de UID ya existente
POST /{id}/bloquear	200 estado=bloqueada	Tarjeta bloqueada con motivo
POST /{id}/desbloquear	200 estado=activa	Tarjeta regresa a activa
DELETE /tarjetas/1	200 o 204	Soft delete confirmado

11. Módulo: Lecturas de Tarjeta

Log de cada uso de tarjeta RFID/NFC en los diferentes módulos del campus (cafetería, copias, biblioteca, acceso, etc.).

Endpoints

Método	URL	Descripción
GET	/tarjeta-lecturas	Historial completo de lecturas
GET	/tarjeta-lecturas?tarjeta_id=1	Lecturas de la tarjeta ID 1
GET	/tarjeta-lecturas?uid_leido=A1B2C3D4	Lecturas de ese UID específico
GET	/tarjeta-lecturas?modulo=cafeteria	Lecturas en cafetería
GET	/tarjeta-lecturas?tipo_lectura=consumo	Solo lecturas de consumo
GET	/tarjeta-lecturas?exito=false	Lecturas fallidas
GET	/tarjeta-lecturas?pedido_id=5	Lecturas del pedido ID 5
GET	/tarjeta-lecturas/{id}	Detalle de lectura específica
POST	/tarjeta-lecturas	Registrar nueva lectura (hardware)

Parámetros de Consulta

Parámetro	Tipo	Descripción
tarjeta_id	integer	ID interno de la tarjeta
uid_leido	string	UID detectado por el lector
modulo	string	cafeteria · copias · souvenirs · biblioteca · acceso · otro
tipo_lectura	string	acceso · consumo · consulta_saldo · confirmacion_entrega
exito	boolean	true = lecturas exitosas · false = fallidas
pedido_id	integer	ID del pedido asociado a la lectura
operador_id	integer	ID del operador que registró la lectura

Ejemplos de Body (JSON)

POST /tarjeta-lecturas – Registrar lectura

```
{
  "uid_leido":      "A1B2C3D4E5F6",
  "modulo":         "cafeteria",
  "tipo_lectura":   "consumo",
  "exito":          true,
  "detalle":        "Lectura en caja 1",
  "operador_usuario_id": 2
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta uid_leido o modulo	{ "message": "Los campos uid_leido y modulo son obligatorios" }
404	Lectura no encontrada	{ "message": "Lectura no encontrada" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /tarjeta-lecturas	200 paginado	Historial completo
GET ?modulo=cafeteria	200 filtrado	Solo lecturas de cafetería
GET ?tipo_lectura=consumo	200 solo consumos	Lecturas de cobro
GET ?exito=false	200 fallidas	UIDs inválidos o tarjetas bloqueadas
GET ?pedido_id=5	200 por pedido	Confirmación de entrega
POST – lectura exitosa	201 Created	Lectura registrada y exito=true
POST – uid de tarjeta bloqueada	201 exito=false	Registro con exito=false automático
GET /tarjeta-lecturas/999	404	Lectura no encontrada

12. Módulo: Saldo Monedero

Monedero digital de cada usuario. Almacena el saldo disponible y el saldo retenido. Se consulta antes de cualquier cobro y se crea automáticamente al registrar un usuario.

Endpoints

Método	URL	Descripción
GET	/saldo-monederos	Todos los monederos del sistema
GET	/saldo-monederos?usuario_id=1	Monedero del usuario ID 1
GET	/saldo-monederos/usuario/1	Consulta directa de saldo (ruta principal)
GET	/saldo-monederos/{id}	Monedero por ID interno
POST	/saldo-monederos	Crear monedero para usuario nuevo

Parámetros de Consulta

Parámetro	Tipo	Descripción
usuario_id	integer	Filtra monedero de un usuario específico

Ejemplos de Body (JSON)

POST /saldo-monederos – Crear monedero

```
{
  "usuario_id": 4
}
```

Casos de Error

Código	Causa	Respuesta esperada
--------	-------	--------------------

400	Falta usuario_id	{ "message": "El campo usuario_id es obligatorio" }
404	Usuario sin monedero registrado	{ "message": "Monedero no encontrado" }
422	Usuario ya tiene monedero	{ "message": "El usuario ya tiene un monedero registrado" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /saldo-monederos/usuario/1	200 saldo disponible	Retorna saldo_disponible y saldo_retenido
GET /saldo-monederos/usuario/999	404	Usuario sin monedero
POST – usuario sin monedero	201 Created	Monedero con saldo 0 creado
POST – usuario con monedero	422	Error: monedero ya existe

13. Módulo: Saldo – Movimientos

Registro de todos los abonos (recargas) y cargos (consumos) al monedero. Actualiza automáticamente el saldo disponible. Rechaza cargos si no hay saldo suficiente.

Endpoints

Método	URL	Descripción
GET	/saldo-movimientos	Todos los movimientos
GET	/saldo-movimientos?usuario_id=1	Estado de cuenta del usuario 1
GET	/saldo-movimientos?tipo=abono	Solo recargas de saldo
GET	/saldo-movimientos?tipo=cargo	Solo consumos descontados
GET	/saldo-movimientos?modulo=cafeteria	Movimientos de cafetería
GET	/saldo-movimientos?desde=X&hasta=Y	Movimientos en rango de fechas
GET	/saldo-movimientos/{id}	Detalle de movimiento específico
POST	/saldo-movimientos	Registrar abono o cargo

Parámetros de Consulta

Parámetro	Tipo	Descripción
usuario_id	integer	Filtra movimientos de un usuario
tipo	string	abono · cargo
modulo	string	cafeteria · copias · souvenirs · biblioteca · recarga · otro
saldo_monedero_id	integer	ID interno del monedero
desde/hasta	date	Rango de fechas YYYY-MM-DD

Ejemplos de Body (JSON)

POST – Abono (recarga de saldo)

```
{
```

```

"usuario_id": 4,
"tipo": "abono",
"monto": 100.00,
"modulo": "recarga",
"concepto": "Recarga en ventanilla",
"operador_usuario_id": 1
}

```

POST – Cargo (consumo)

```

{
  "usuario_id": 4,
  "tipo": "cargo",
  "monto": 35.50,
  "modulo": "cafeteria",
  "concepto": "Comida del dia",
  "operador_usuario_id": 2,
  "tarjeta_lectura_id": 1,
  "referencia_tabla": "pedido",
  "referencia_id": 5
}

```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta tipo, monto o usuario_id	{ "message": "Campos obligatorios faltantes" }
404	Usuario o monedero no existe	{ "message": "No se encontró el monedero del usuario" }
422	Saldo insuficiente para el cargo	{ "message": "Saldo insuficiente", "saldo_disponible": 10.00 }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /saldo-	200 estado de	Lista de abonos y cargos del usuario

movimientos?usuario_id=1	cuenta	
GET ?tipo=abono	200 solo recargas	Filtrado por tipo abono
GET ?modulo=cafeteria	200 por módulo	Movimientos originados en cafetería
POST – abono \$100	201 saldo actualizado	Saldo_disponible incrementado en \$100
POST – cargo con saldo suficiente	201 saldo descontado	Incluye saldo_antes y saldo_despues
POST – cargo sin saldo suficiente	422 saldo insuficiente	Error con saldo_disponible actual
POST – cargo monto negativo	400 Bad Request	Validación: monto debe ser positivo
GET /saldo-movimientos/1	200 detalle	Incluye monto, saldo_antes, saldo_despues, concepto

14. Módulo: Pedidos

Gestión del ciclo de vida de pedidos: creado → aceptado → en_proceso → listo → entregado. Incluye confirmación por tarjeta RFID/NFC para la entrega.

Endpoints

Método	URL	Descripción
GET	/pedidos	Todos los pedidos del sistema
GET	/pedidos?usuario_id=4	Pedidos del usuario ID 4
GET	/pedidos?estado=creado	Pedidos nuevos entrantes
GET	/pedidos?estado=listo	Pedidos listos para entregar
GET	/pedidos?modulo=cafeteria	Pedidos de cafetería
GET	/pedidos?folio=F-ABC	Búsqueda por folio
GET	/pedidos?desde=X&hasta=Y	Pedidos en rango de fechas
GET	/pedidos/{id}	Detalle completo de un pedido
POST	/pedidos	Crear pedido (checkout)
PUT	/pedidos/{id}	Actualizar notas o descripción
DELETE	/pedidos/{id}	Baja lógica del pedido
POST	/pedidos/{id}/estado	Avanzar estado del pedido
POST	/pedidos/{id}/confirmar-tarjeta	Confirmar entrega con tarjeta RFID

Parámetros de Consulta

Parámetro	Tipo	Descripción
usuario_id	integer	Pedidos de un usuario específico
estado	string	creado · aceptado · en_proceso · listo · entregado · cancelado
modulo	string	cafeteria · copias · souvenirs · biblioteca · otro
operador_id	integer	Pedidos atendidos por un operador
folio	string	Búsqueda parcial de folio

desde/hasta	date	Rango de fechas YYYY-MM-DD
-------------	------	----------------------------

Ejemplos de Body (JSON)

POST /pedidos – Crear pedido

```
{
  "usuario_id": 4,
  "modulo": "cafeteria",
  "total": 35.50,
  "descripcion": "Comida del día: arroz, pollo y agua",
  "notas": "Sin chile"
}
```

POST /pedidos/{id}/estado – Avanzar estado

```
{
  "estado": "aceptado",
  "operador_usuario_id": 2
}
```

POST /pedidos/{id}/confirmar-tarjeta – Confirmar entrega

```
{
  "tarjeta_lectura_id": 5
}
```

Casos de Error

Código	Causa	Respuesta esperada
400	Falta usuario_id, modulo o total	{ "message": "Campos requeridos faltantes" }
404	Pedido no encontrado	{ "message": "Pedido no encontrado" }
422	Estado de transición inválido	{ "message": "No se puede pasar de entregado a creado" }
422	Lectura de tarjeta no corresponde	{ "message": "La lectura no está asociada a este pedido" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
GET /pedidos?estado=creado	200 pedidos nuevos	Panel del proveedor: pedidos entrantes
GET /pedidos?estado=listo	200 listos para entrega	Panel: pedidos listos
GET /pedidos?modulo=cafeteria	200 filtrado por módulo	Solo pedidos de cafetería
POST /pedidos – body válido	201 folio generado	Retorna folio único autogenerado
POST /pedidos – sin usuario_id	400 Bad Request	Validación de campo requerido
POST /{id}/estado – flujo normal	200 estado actualizado	Estado avanza correctamente
POST /{id}/estado – estado inválido	422	Transición no permitida rechazada
POST /{id}/confirmar-tarjeta – válido	200 entregado	Estado = entregado, lectura asociada
POST /{id}/confirmar-tarjeta – mal ID	422	Lectura no corresponde al pedido
DELETE /pedidos/1	200 o 204	Baja lógica del pedido
GET /pedidos/999	404	Pedido no encontrado

15. Módulo: RFID – Autenticación y Consulta

Endpoints especializados para hardware y sistemas externos. Permiten autenticar usuarios por tarjeta, consultar saldo, historial y pedidos sin necesidad de la sesión web. La ruta /rfid/auth es pública (no requiere API Key).

Endpoints

Método	URL	Descripción
POST	/rfid/auth	Autenticar usuario con UID + PIN (pública)
POST	/rfid/verificar	Verificar validez de UID + PIN
GET	/rfid/usuario/{uid}	Perfil completo del dueño de la tarjeta
GET	/rfid/saldo/{uid}	Saldo del monedero por UID
GET	/rfid/historial/{uid}	Historial de movimientos por UID
GET	/rfid/pedidos/{uid}	Pedidos activos por UID
GET	/rfid/lecturas/{uid}	Historial de lecturas por UID

Parámetros de Consulta

Parámetro	Tipo	Descripción
tipo	string	abono · cargo (en historial)
modulo	string	cafeteria · copias · etc. (en historial y pedidos)
estado	string	Estado de pedido para filtrar en /rfid/pedidos/{uid}
tipo_lectura	string	acceso · consumo · etc. (en lecturas)
desde/hasta	date	Rango de fechas para lecturas
per_page	integer	Registros por página

Ejemplos de Body (JSON)

POST /rfid/auth – Autenticar (pública)

```
{
  "uid": "8FE785CD",
  "pin": "1234"
}
```

POST /rfid/verificar – Verificar sin datos

```
{
  "uid": "A1B2C3D4",
  "pin": "1234"
}
```

Casos de Error

Código	Causa	Respuesta esperada
401	API Key inválida (excepto /rfid/auth)	{ "message": "Unauthorized" }
401	PIN incorrecto en /rfid/auth	{ "message": "PIN inválido" }
404	UID no registrado en el sistema	{ "message": "Tarjeta no encontrada" }
403	Tarjeta bloqueada o cancelada	{ "message": "Tarjeta bloqueada" }

Casos de Prueba

Escenario	Resultado Esperado	Validación
POST /rfid/auth – UID+PIN válidos	200 datos completos	Devuelve usuario, roles, estado tarjeta, saldo
POST /rfid/auth – PIN incorrecto	401 Unauthorized	PIN inválido sin exponer datos
POST /rfid/auth – UID inexistente	404	Tarjeta no encontrada
POST /rfid/auth – sin API Key	200 (es pública)	Ruta pública, no requiere X-API-KEY
POST /rfid/verificar – válido	200 { ok, valido, uid }	Solo booleano, sin datos sensibles
GET /rfid/saldo/A1B2C3D4	200 saldo_disponible	Nombre + saldo para mostrar en pantalla

GET /rfid/historial/{uid}?tipo=cargo	200 solo cargos	Historial filtrado por tipo
GET /rfid/pedidos/{uid}?estado=listo	200 pedidos listos	Pedidos que puede recoger ahora
GET /rfid/pedidos/{uid}?estado=listo&modulo=cafeteria	200 combinado	Filtro doble: listo + cafetería
GET /rfid/lecturas/{uid}?modulo=cafeteria	200 lecturas en cafetería	Historial de uso en módulo cafetería
GET /rfid/usuario/UID_INVALIDO	404	UID no registrado
GET /rfid/saldo/{uid} tarjeta bloqueada	403 o datos con alerta	Indicar estado de tarjeta bloqueada

